Distributed and Mobile Computing Systems
D7024E

Distributed consensus

Olov Schelén

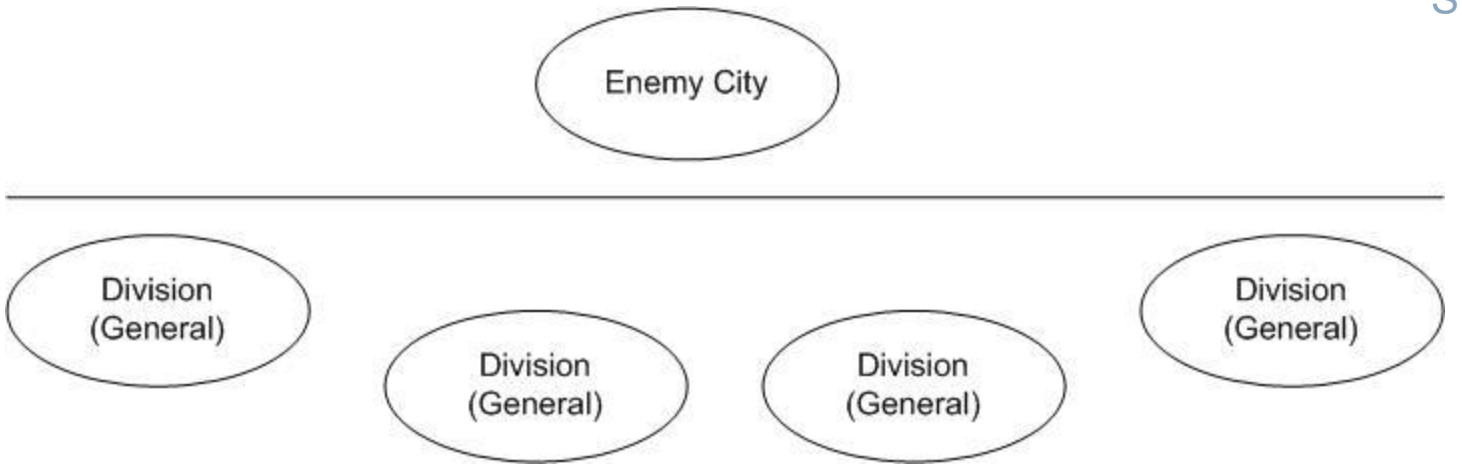


Today's lecture

- Byzantine fault tolerance (consensus & traitors)
- Critical assumptions in distributed systems
- Global state synchronization, basics
  - Case study: Paxos
- Global state synchronization with leaders and epochs
  - Case study: Zookeeper, Zab
  - Case study Raft, (Etcd)
- Other approaches for leader elections
  - Ring-based (by position & token)
  - Bully (by ordered identities)

Byzantine Generals, problem

Modified slides from
Source:
Anthony Soo Kaim
Ryan Chu
Stephen Wu



- Divisions of the Byzantine army camped outside the walls of an enemy city.
- Each division is led by a general.
- Generals decide on a common plan of action
  - Either ATTACK or RETREAT

Problem & conditions

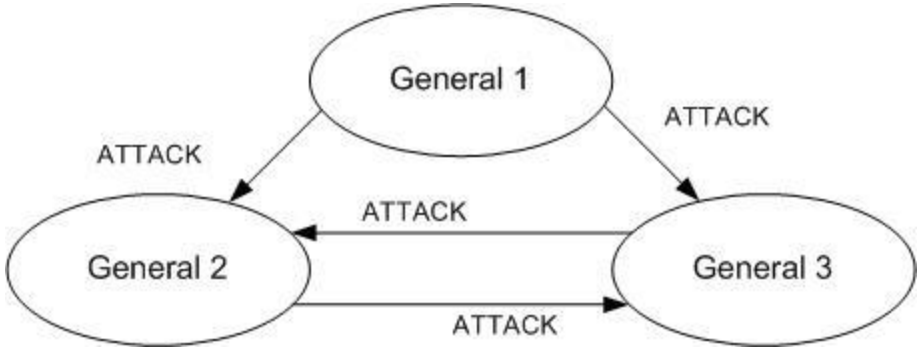
- There are two types of generals
  1. Loyal Generals – repeating what they heard
  2. Traitor Generals – lying about what they heard
- Two safety conditions MUST be met
  1. All loyal generals decide upon the same plan of action.
  2. A small number of traitors must not cause the loyal generals to adopt a bad plan.

Liveness (reaching an agreement) is typically not guaranteed but should happen statistically after some rounds.

Agreeing on a Plan
Obtaining consensus

- A general may initiate an action v (attack or retreat)
- Agreement is obtained in the following way:
  - Each general sends (repeats) his observation to all other generals.
  - Let v(i) be the message communicated by the ith general.
  - The combination of the v(i) for i = 1, …, n messages received determine the agreed plan.
- Any two loyal generals use the same value of v(i).
- If the ith general is loyal, then the value that he sends must be used by every loyal general as the value of v(i).

Example Agreed Plan
(initiated by General 1)

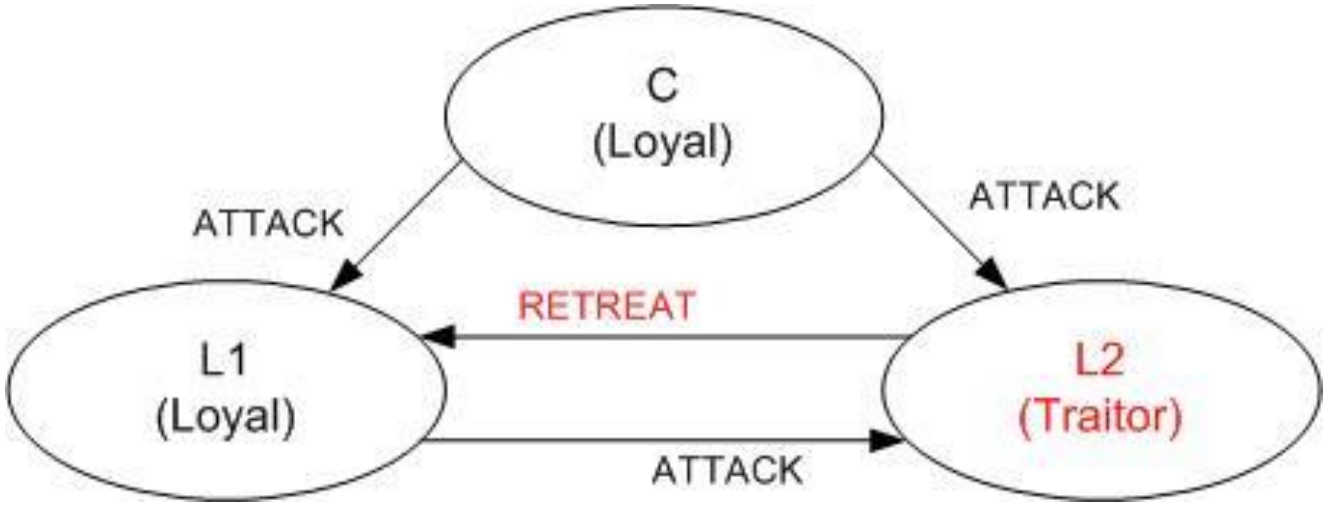


- General 2 receives ATTACK, ATTACK.
- General 3 receives ATTACK, ATTACK.
  - So agreed Plan is to ATTACK

Problem – Traitor(s)

- So, every general should communicate the same v(i) to every other general.
- However, a traitor general can lie and send different v(i) messages to different generals.
- Messages are only seen by pairwise sender-receiver (other parties cannot see them)

Example Traitor
(command initiated by C)



- L1 received the commands ATTACK, RETREAT
- L1 doesn't know the plan or which general is a traitor.

Limitations on reaching consensus in the pairwise messaging model

- In previous example the system will stall
  - No consensus can be reached, because of one traitor
  - When will the Byzantine Generals fail to reach consensus?
- They will fail if 1/3 or more of the generals are traitors.

Generalization of the limitations

- No consensus in the pairwise messaging model when:
  - Fewer than 3m + 1 generals;
    - m = number of traitor generals
  - Other words: There is no solution if 1/3 or more of the nodes are traitors
- This is under the assumptions
  - all messages arrive
  - each party knows the number of other parties and their identities
- If a system is byzantine fault-tolerant, the system is resilient to some minority of traitors (malicious nodes) that deliberately try to fool the system
  - Note that most global state algorithms are non-Byzantine, and most distributed systems are non-Byzantine (i.e., honest crash/clash-tolerant only)

What about messages being forged or lost along the way?
Must be prevented…



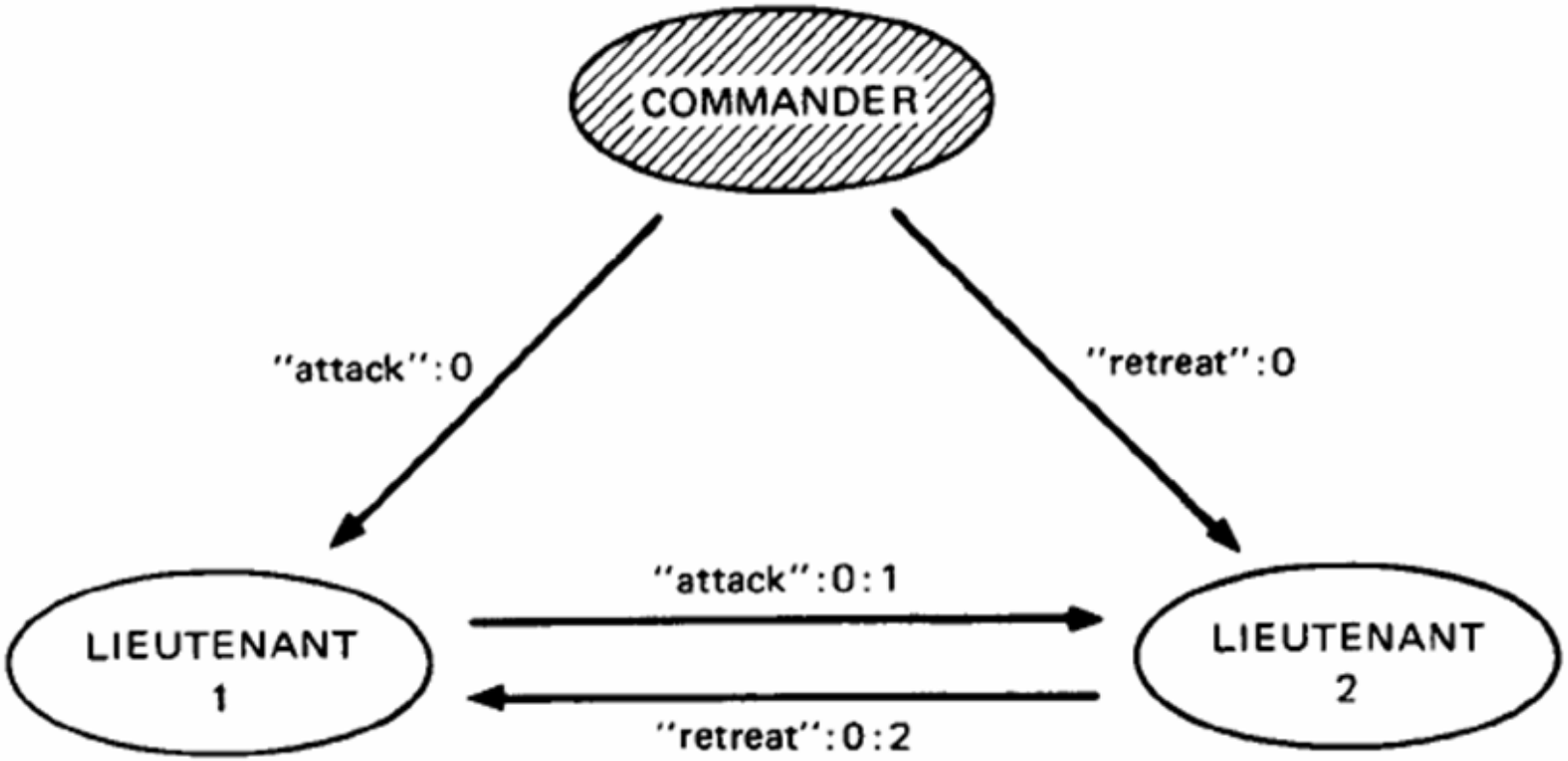
Solution with Signed Messages

- Require generals to send and forward unforgeable, signed messages
- Assumptions:
  - **A1**: A message that is sent is not corrupted.
  - **A2**: The receiver of a message knows who sent it.
  - **A3**: The absence of a message can be detected.
  - **A4**: Signatures
    a) A loyal general's signature cannot be forged, and any alteration of the contents of his signed messages can be detected.
    b) Anyone can verify the authenticity of a general's signature.

Solution with Signed Messages

- Notation for signed messages:
  - x : i denotes the value x is signed by General i
  - x : i : j denotes x is signed by i, and x : i is then signed by j

  - Each lieutenant maintains a set V, containing the set of properly signed orders he has received so far



- Both L1 and L2 can conclude that the commander is a traitor

Reliable messaging

- “Easy” to implement the algorithm inside a computer (because messages are not lost or compromised)
- Problem is to handle lost messages
  - A) by making an algorithm that can handle outages
  - B) or, by providing a reliable *message passing system*

A reliable messaging system would have to ensure the following assumptions

- ***A1**: Every message sent by a non-faulty node is delivered correctly.*
- *A2: A node can determine the originator of the message received.*
  - Means a faulty node cannot impersonate a non-faulty one. By signatures or by trust
- *A3: The absence of a message can be detected.*
  - *By timeouts and by sequencing/ordering*

NOTE that many global consensus solutions are built on top of a reliable messaging service

Byzantine consensus is very complex

- We just touched the surface on the problem with traitors
  - What if commander crashes? What if anyone can attempt to be commander? We need to support elections and re-tries (or similar methods), which imposes a harder consensus problem.
- We will get back to byzantine consensus in later lectures
  - e.g., part of blockchains
- Now, we first need to understand **Non-Byzantine** consensus
  - Hard enough when honest nodes can crash

Global consensus by Paxos (synchronization of STATE)

Sources:

- Leslie Lamport, Jim Gray, Ion Stoica.
- Shostak, Pease, & Lamport, JACM, 1980

Modified by Olov Schelén



Paxos objective

- To reach consensus/data consistency, e.g. agreement on a common “value” in distributed system that can tolerate non-malicious failures
  - Nodes do follow an agreed protocol to reach consensus,
  - but individual nodes may crash or be unreachable (e.g., due to network failures).
- This system is non-Byzantine
  - Any single malicious node (traitor) could make sure that consensus is never reached

Attempt: Two phase commit (2PC) (one leader and many participants)

- **Proposal-Voting Phase** – A coordinator/leader suggests a value to all Resource Managers and gathers their responses (whether they agree to the value or not). The RMs reply with a vote: yes or no.
  - Note that the vote is only on the proposed value – a node can only say yes or no. It cannot suggest an alternative.
- **Commit Phase** - If nodes agree, the leader contacts all nodes to let them know the value is final.
  - If even one node does not agree, inform all nodes that the value is not final.
  - If not final, a new round must be started

Properties of "basic" 2PC with a permanent distinguished leader

- Atomicity – all or nothing
- Consistency – does right thing
- Isolation – no concurrency anomalies
- Durability / Reliability – state survives failures
- Availability: always up



Blocks if leader fails (crash, disconnect)

Alternative approaches

- Extended 2 phase: 1a, 1b, 2a, 2b (sometimes denoted 3 phase) and commit leader for every round
  - Example: Paxos
- Dynamic leader election in epochs
  - Extended 2 phase to elect a leader
  - A leader stays leader for many rounds (an epoch)
  - Examples: Zookeeper, Raft

Common requirements in distributed systems

- Safety
  - Only a value that has been proposed may be chosen.
  - Only a single value is chosen.
  - A node never learns that a value has been chosen unless it actually has been.
  - Can safety be guaranteed?
- Liveness
  - Some proposed value is eventually chosen.
  - If a value has been chosen, a node can eventually learn the value
  - Can liveness be guaranteed?

Paxos Properties

- Paxos is guaranteed safe.
  - Consensus is a stable property: once reached it is never violated; the agreed value is not changed.

- Paxos is not guaranteed live.
  - Consensus is reached if “a large enough subnetwork...is non-faulty for a long enough time.”
  - Otherwise Paxos might never terminate. Actually, typically true for any consensus algorithm

Paxos Properties

- Paxos is an asynchronous consensus algorithm
  - Asynchronous networks
    - No common clocks or shared notion of time (local ideas of time are fine, but different processes may have very different “clocks”)
    - No way to know how long a message will take to get from A to B

Paxos Algorithm

- Key Assumptions:
  - Set of processes that run Paxos is known a-priori
    - Note: it is not possible to reach consensus (to be safe) if we don’t know who or how many of us that are to be involved in the consensus decision.
  - Processes suffer crash failures
    - i.e., non-byzantine (there are no traitors that deliberately sabotage the outcome)

Paxos rounds and roles

- Each round has a unique number (monotonically increasing), driven by ONE leader node (proposer), but the leader may vary for each round

- Rounds are asynchronous
  - Time synchronization not required
  - If you are in round j and hear a message from round j+1, abort everything and go to round j+1
  - All nodes are cooperative and will try to avoid creating “competing” rounds (avoid race conditions as far as they can)

- There are 3 roles for the nodes
  - proposer
  - acceptor
  - learner
    - A node can act as more than one role (usually 2 or 3).

Paxos Algorithm

- 2*2 phases
  - Phase 1a: Prepare request (leader -> acceptors)
  - Phase 1b: Promise (acceptors -> leader)

  - Phase 2a: Accept request (leader -> acceptors)
  - Phase 2b: Accepted/Decide (acceptors->learners) (Note: leader is also learner)

Combination of roles

- Over time, every node is acceptor, proposer, and learner
- Any node can initialize to lead a round
  - an accepted leader is the distinguished proposer and a learner
  - other nodes noticing the ongoing round will avoid dueling for the proposer role
- The leader is therefore selected for each round

Phase 1: (prepare request)

(a) A proposer chooses a new round number n , and sends a prepare request (“prepare”,n) to a majority of acceptors:

(a) Can I make a proposal with number n ?

(b) Waits for promises (yes-answers), possibly with more info.

Note that any node can try to be a leader (proposer) when they have a new value (which could happen “simultaneously” within propagation delays). They may be on the same n or different n (compare with the virtual clock)

(b) An acceptor that receives a prepare request (“prepare”, n) with n greater than in those prepare request it has already acked, sends out

- (“ack”, n, n',v'), i.e., a promise not to accept any more proposals numbered less than n, AND informs about the previously highest round n' and its value v' (useful in case the new value must depend on earlier value)

Note: if the proposal has a lower round n than earlier seen, there is no ack sent (or alternatively a NAK may be sent for optimization)

Phase 2: (leader collects votes and issues a request to accept the voting outcome)

(a) If the proposer receives responses for the n from a majority of the acceptors, then it can issue an accept request (“accept”, n , v) with number n and value v:

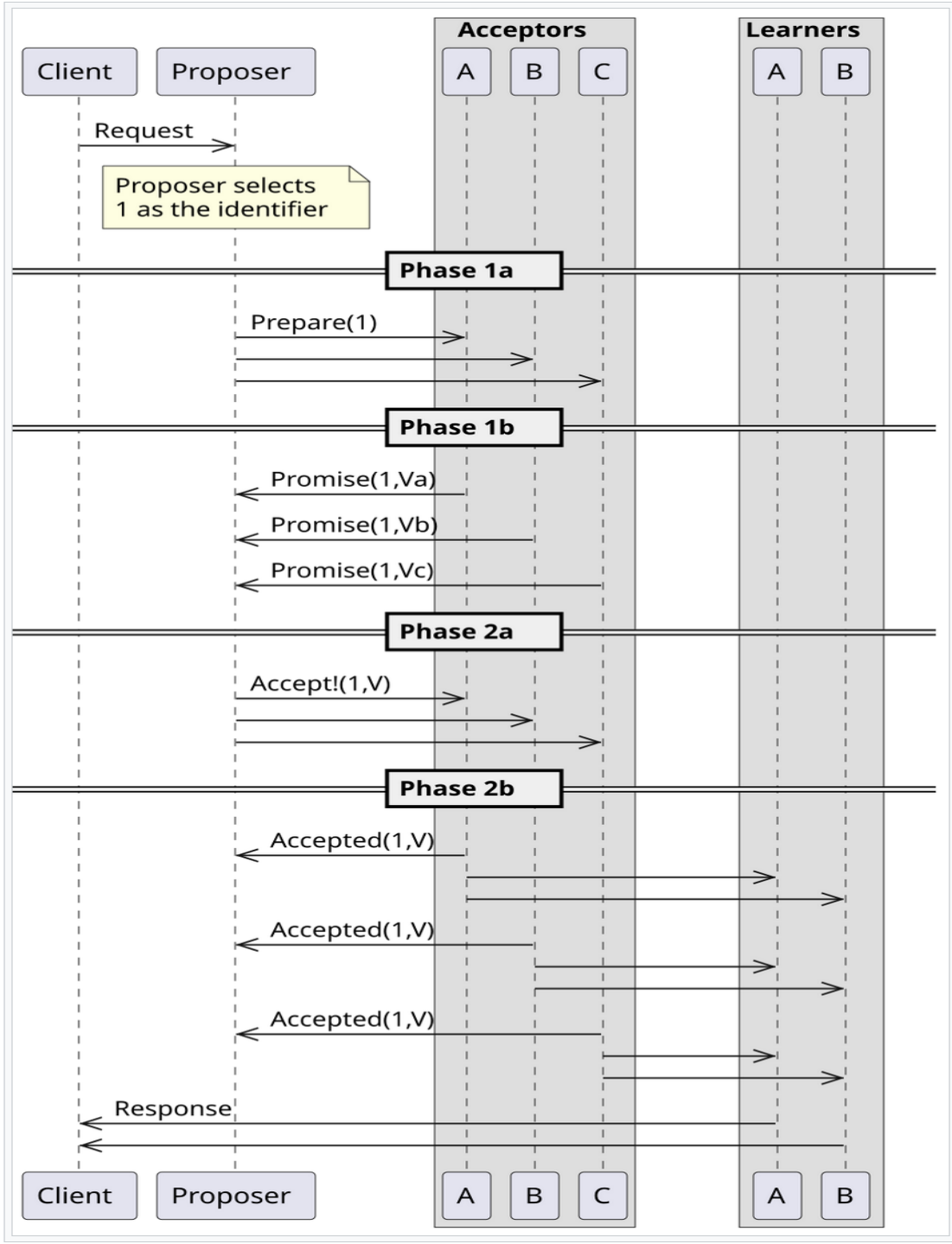
(b) If the acceptor receives an accept request from the leader (“accept”, n , v) , it accepts the proposal by sending an "accepted", unless it has already responded to a prepare request having a number greater than n (which may have happened in the meantime). An "Accepted"b is sent to all other parties (listeners).

When a party sees a majority of Accepted, it is a fact thar a majorty of acceptors have finally agreed so it is final even if some listener for the moment couldn't get the message

See wikipedia

https://en.wikipedia.org/wiki/Paxos_(computer_science)

Note: Accepted should be sent to all, including the acceptors, which are also learners (dual roles)



Many error cases to handle

- Can handle if a minority of acceptors or learners fail
- If a leader fails during a round, another leader will try to take over
  - Randomized hold timers may trigger when other nodes try to be leader
- Dueling leaders (e.g., due to delay) will be detected (not leading to decision)
  - A new round must be started, but not simultaneously again
  - Randomized hold timers should ensure that eventually there is a round with one designated leader

Paxos number of nodes

- Lots of messages! M ≥ 4(N-1) for each successful round
  - N-1 prepare, N-1 promise, N-1 accept, N-1 accepted
  - Plus retries when consensus is not reached
- N >= 2F+1 acceptors (i.e., 3, 5, 7… nodes needed)
  - Where F is the max resilience to concurrently failing nodes
- Essential to have as reliable nodes as possible

Applications

- Chubby lock service.
- Petal: Distributed virtual disks.
- Frangipani: A scalable distributed file system.

Apache ZooKeeper

References:

- Mahadev Konar (Yahoo!)
- Saurav Haloi (Symantec)
- Saurav Haloi (Yahoo) GNUnify 2013 talk



Coordination in a distributed system

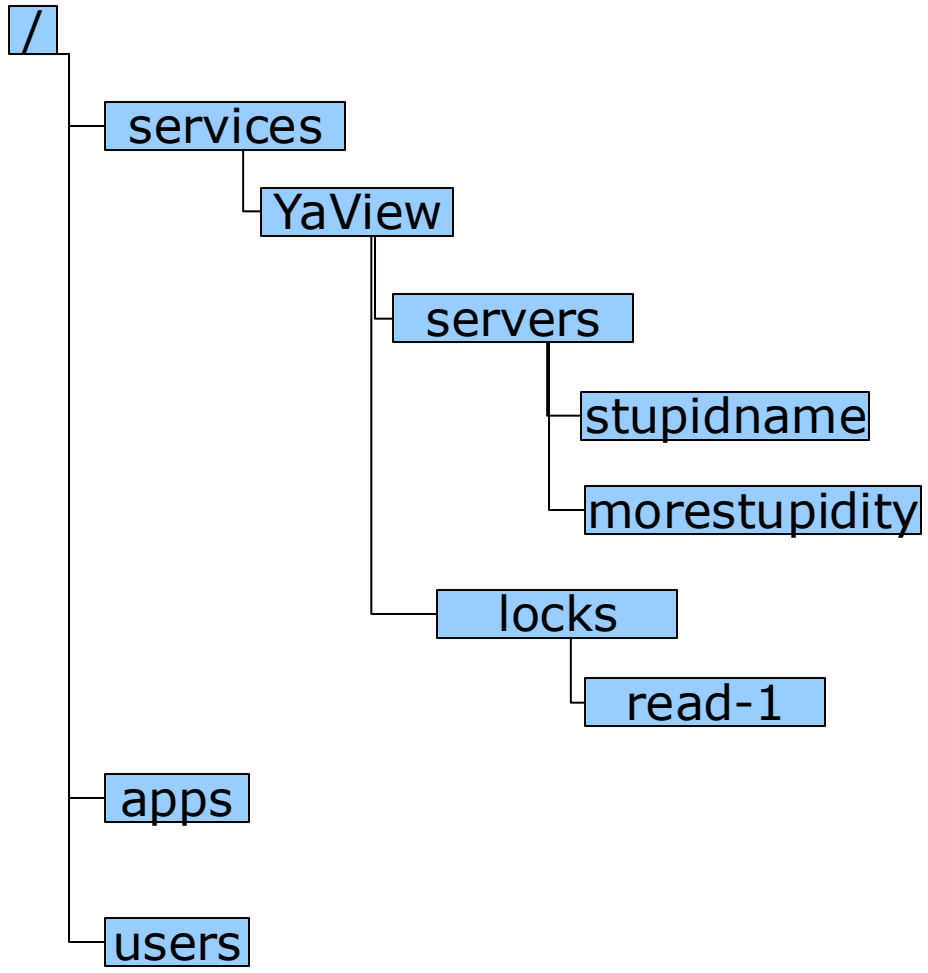
- Multiple autonomous nodes must coordinate on the content of some shared state
- Examples
  - Group membership
  - Leader election
  - Synchronization
- Under network conditions with delay and (partial) outages this is hard!

What is Zookeeper?

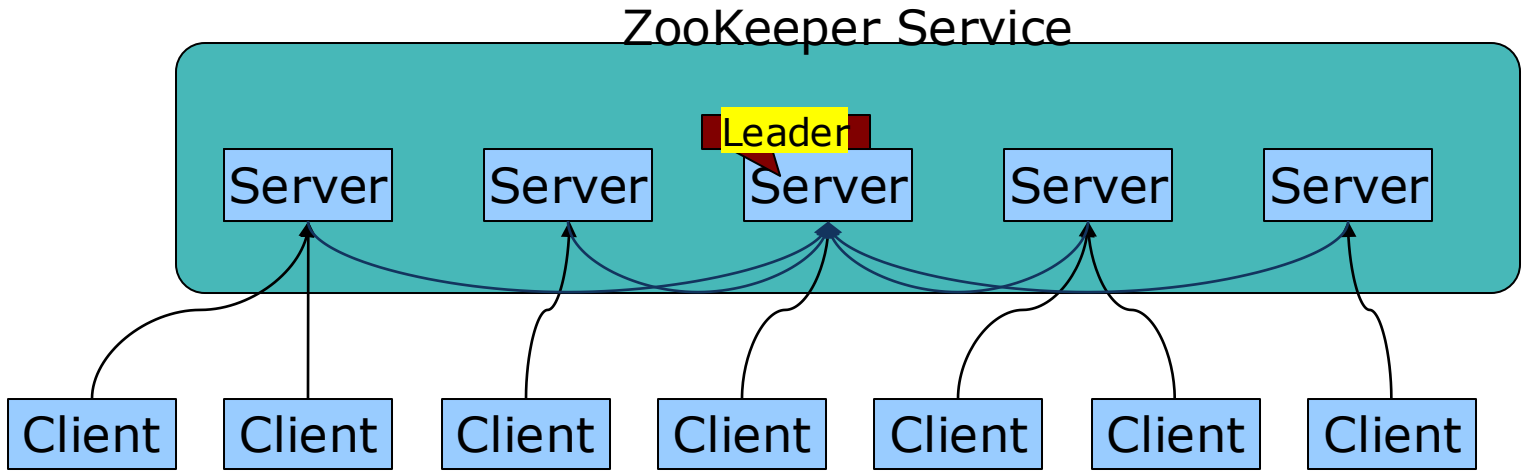
- A highly available, scalable, distributed, configuration, consensus, group membership, leader election, naming, and coordination service
- Zookeeper provides a shared hierarchical namespace of data registers
- This is more powerful than a distributed lock server

Data Model

- Hierarchal namespace represented by znodes
  - Each znode has data and children znodes
- Data is read and written for a given znode in its entirety (atomically)
- Each znode has an Access Control List (ACL) that restricts who can do what



ZooKeeper Service



- ZooKeeper (ZK) service is replicated over a set of machines/servers
- All ZK servers store a copy of the data (in memory)
- A leader is elected at startup (and re-elected at failures)
- Clients connect to a single ZK server (single point of contact (SPOC)) and maintains a TCP connection
- Clients read directly from any ZK server (i.e, the one they picked for SPOC)
- However, all updates go through leader (forwarded by ZK servers)
- Update responses are sent when a majority of servers have persisted the change

Analysis

- Both fault tolerance and read throughput scales by adding servers
- Write throughput does not scale by adding servers (actually it may be a bit on the contrary)
  - all writes must go to the single leader and then be sent to all nodes for reaching consistency
  - write throughput it is limited by performance of this
- Zookeeper uses an OTT broadcast protocol named Zab
  - All broadcast actions are **idempotent**

Zab broadcast guarantees

- Reliable (atomic) delivery: If a message, m, is delivered to one server, then it will be eventually delivered to all correct servers.
- Total order: If message a is delivered before message b to one server, then every server that receives a and b will receive a before b.
- Prefix property: If m is the last message delivered for a leader L, any message proposed before m by L must also be delivered;
  - Note that a process might be elected multiple times. Each time, however, counts as a different leader for the purposes of this prefix property.

Zab

- primary-order atomic broadcast, or simply PO atomic broadcast
  - The protocol proceeds in epochs, and each epoch can have at most one process broadcasting messages (i.e., new leader = new epoch)
  - Once a process executes the first phase of the protocol, it promises to not accept proposals from previous epochs. This property is critical to guarantee that during recovery no chosen proposal is missing from the selected initial history of transactions;
  - For a given epoch, all processes that participate in the epoch must have the initial history of transactions as a prefix of its own history
  - To guarantee that transactions are generated using a consistent state, we need to make sure that a process has completed recovery before it broadcasts new transactions.

Zookeeper consequences

- With these three guarantees we can maintain correct replicas of the ZooKeeper database:
  - The reliability and total order guarantees ensure that all of the replicas have a consistent state;
  - The causal order ensures that the replicas have state correct from the perspective of the application using Zab;
  - The leader proposes updates to the database based on requests received.
- It is important to observe that there are two types of causal relationships taken into account by Zab:
  - If two messages, a and b, are sent by the same server and a is proposed before b, we say that a causally precedes b;
  - Zab assumes a single leader server at a time that can commit proposals. If a leader changes, any previously proposed messages causally precede messages proposed by the new leader.

Zab vs Paxos

- Paxos selects leader in each round and handles message losses and reordering of messages, all at once
  - Requires three phases for each round and if the network is shaky, it will may need lots of rounds and lots of messages
- Zab uses the three-phase only to select a persistent leader, and uses TCP to communicate between pairs of servers, which guarantees that delivered messages are delivered in FIFO order
  - By epochs, provides per proposer causality

Zookeeper internal leader activation

- Leader requirements:
  - The leader has seen the highest zxid of all the followers. Hard requirement.
  - A quorum of servers have committed to following the leader. Soft requirement, restart election if failing
- After successful leader election a single server will wait for followers to connect.
  - The leader will sync up with followers by sending any proposals they are missing, or if a follower is missing too many proposals, it will send a full snapshot of the state to the follower.

Corner case

- A follower that has proposals, U, not seen by a new leader that arrives.
  - Proposals are seen in order, so the proposals of U will have a zxids higher than zxids seen by the leader. The follower had temporary outage during leader election, otherwise the follower would have been elected leader given that it has seen a higher zxid.
  - Since committed proposals must be seen by a quorum of servers, and a quorum of servers that elected the leader did not see U, the proposals have not been committed, so they can be discarded. When the follower connects to the leader, the leader will tell the follower to discard U.

New leader, new epoch

- A new leader establishes a zxid to start using for new proposals by getting the epoch, e, of the highest zxid it has seen and setting the next zxid to use to be (e+1, 0), after the leader syncs with a follower, it will propose a NEW_LEADER proposal.
- Once the NEW_LEADER proposal has been committed, the leader will activate and start receiving and issuing proposals.

Leader activation summary

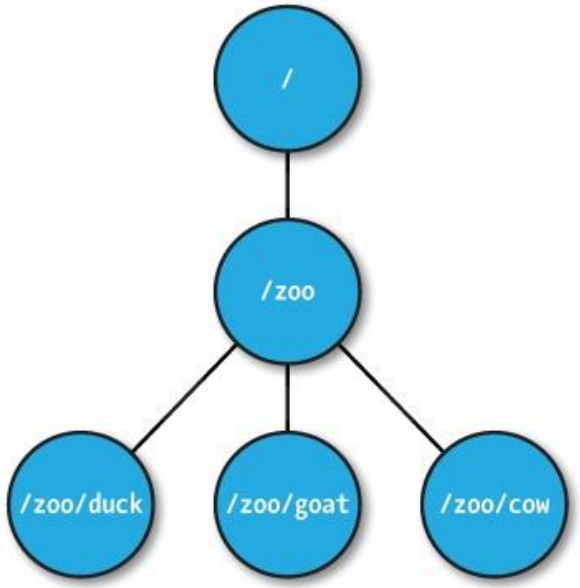
- A follower will ACK the NEW_LEADER proposal after it has synced with the leader.
- A follower will only ACK a NEW_LEADER proposal with a given zxid from a single server.
- A new leader will COMMIT the NEW_LEADER proposal when a quorum of followers have ACKed it.
- A follower will commit any state it received from the leader when the NEW_LEADER proposal is COMMIT.
- A new leader will not accept new proposals until the NEW_LEADER proposal has been COMMITED.
- If leader election terminates erroneously, we don't have a problem since the NEW_LEADER proposal will not be committed since the leader will not have quorum. When this happens, the leader and any remaining followers will timeout and go back to leader election.

Zookeeper cluster setup

- https://zookeeper.apache.org/doc/r3.1.2/zookeeperAdmin.html
  - See clustered operation
  - See transaction log
- Geo distributed zookeeper, yes or no?
  - https://www.quora.com/Has-anyone-deployed-a-ZooKeeper-ensemble-across-data-centers
- Based on ZAB
  - http://diyhpl.us/~bryan/papers2/distributed/distributed-systems/zab.totally-ordered-broadcast-protocol.2008.pdf

More on ZNodes
(this slide and the following 4 are extra readings)

- Contain a stat structure with version numbers for data changes, ACL changes and timestamps.
- Version numbers increases with changes
- Data is read and written in its entirety



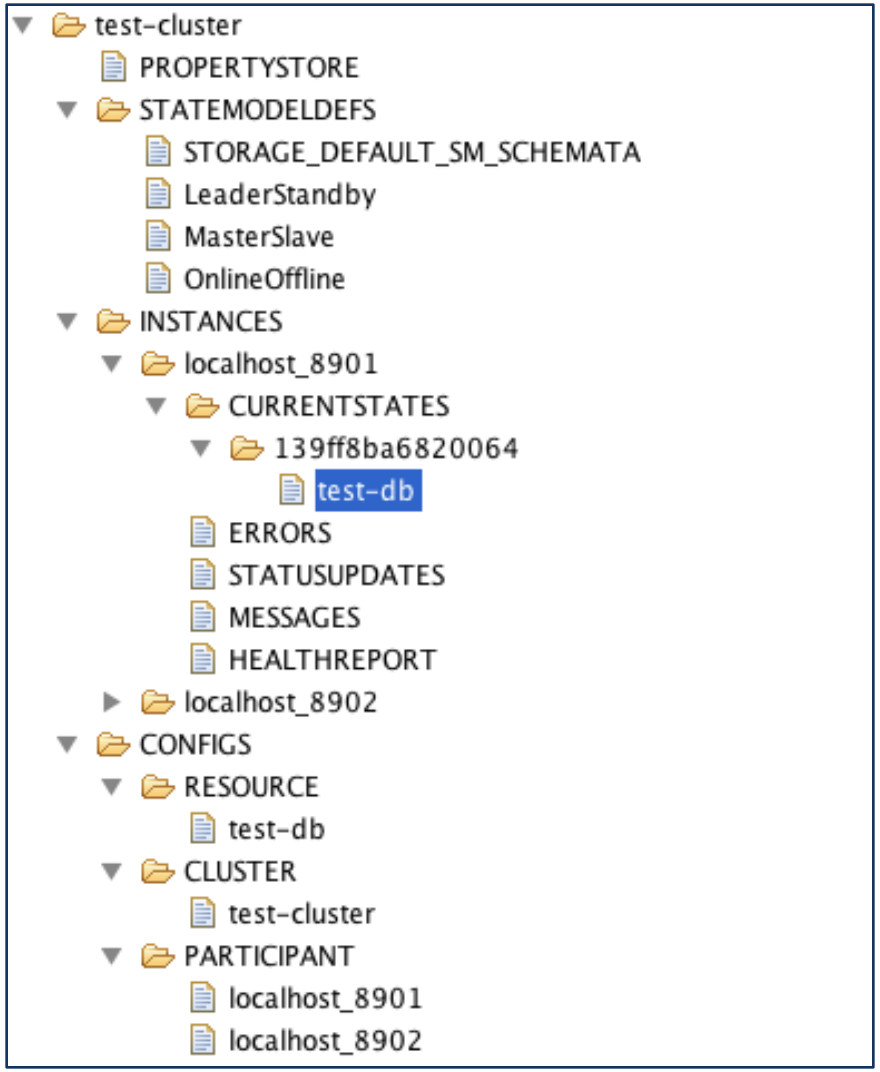


Image: http://helix.incubator.apache.org/Architecture.html

ZNode Types

- Persistent Nodes
  - exists till explicitly deleted
- Ephemeral Nodes
  - exists as long as the session is active
  - can’t have children
- Sequence Nodes (Unique Naming)
  - append a monotonically increasing counter to the end of path
  - applies to both persistent & ephemeral nodes

ZNode Operations

Operation	Type
create	Write
delete	Write
exists	Read
getChildren	Read
getData	Read
setData	Write
getACL	Read
setACL	Write
sync	Read

ZNodes are the main entity that a programmer access.

ZNode Watches

- Clients can set watches on znodes:
  - NodeChildrenChanged
  - NodeCreated
  - NodeDataChanged
  - NodeDeleted
- Changes to a znode trigger the watch and ZooKeeper sends the client a notification.
- Watches are one time triggers.
- Watches are always ordered.
- Client sees watched event before new znode data.
- Client should handle cases of latency between getting the

A few points to remember

- Watches are one-time triggers
  - Continuous watching on znodes requires reset of watches after every events / triggers
- Too many watches on a single znode creates the "*herd effect*" - causing bursts of traffic and limiting scalability
- If a znode changes multiple times between getting the event and setting the watch again, carefully handle it!
- Keep session time-outs long enough to handle long garbage-collection pauses in applications.
- Set Java max heap size correctly to *avoid swapping.*
- Dedicated disk for ZooKeeper transaction log

Language Bindings

- ZooKeeper ships client libraries in:
  - Java
  - C
  - Perl
  - Python
- Community contributed client bindings available for Scala, C#, Node.js, Ruby, Erlang, Go, Haskell

  https://cwiki.apache.org/ZOOKEEPER/zkclientbindings.html

Who uses ZooKeeper?

Companies:

- Yahoo!
- Zynga
- Rackspace
- LinkedIn
- Netflix
- *and many more…*

Open-source projects:

- Apache Map/Reduce (Yarn)
- Apache HBase
- Apache Solr
- Neo4j
- Katta
- *and many more…*

Reference:
https://cwiki.apache.org/confluence/display/ZOOKEEPER/PoweredBy

Reflections on global consensus (e.g., Zookeeper)

- How many nodes should you have?
- What kind of data should you store there?

References

- The Chubby lock service for loosely-coupled distributed systems

  Google Research (7th USENIX Symposium on Operating Systems Design and Implementation (OSDI), {USENIX} (2006))

- ZooKeeper: Wait-free coordination for Internet-scale systems

  Yahoo Research (USENIX Annual Technology Conference 2010)

- Apache ZooKeeper Home: http://zookeeper.apache.org/

- Presentations:
  - http://www.slideshare.net/mumrah/introduction-to-zookeeper-trihug-may-22-2012
  - http://www.slideshare.net/scottleber/apache-zookeeper
  - https://cwiki.apache.org/confluence/display/ZOOKEEPER/ZooKeeperPresentations

Now, we will compare this with raft

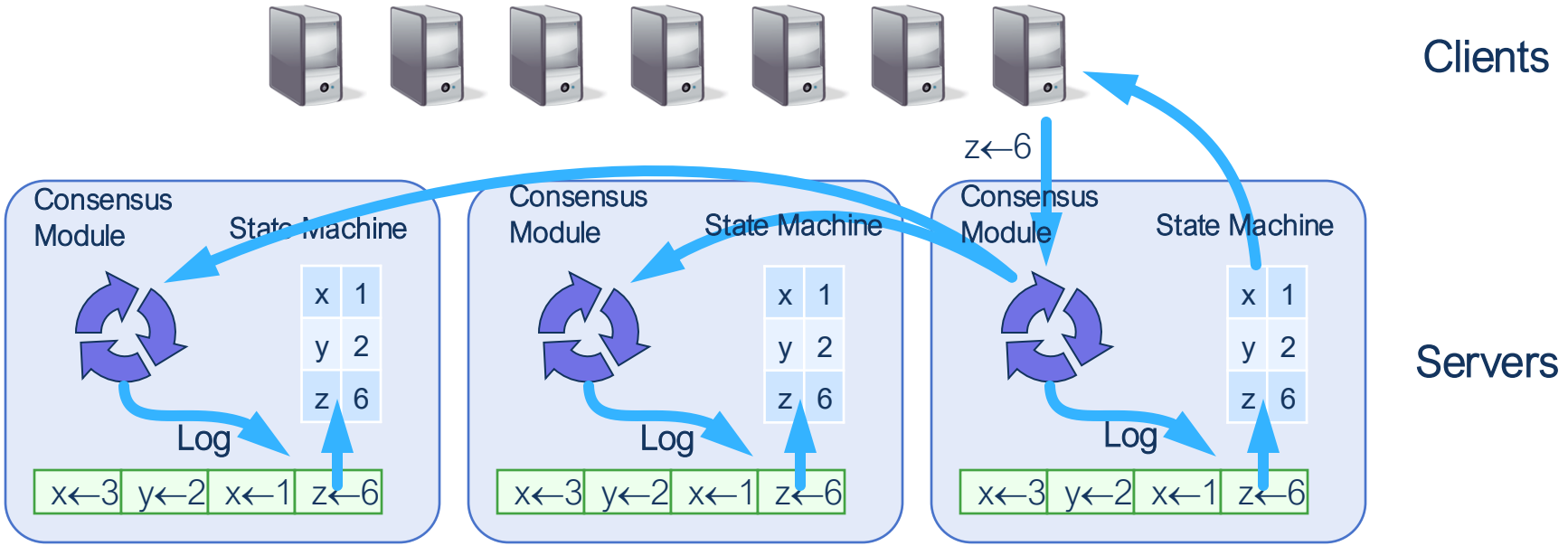
Sources: Ion Stoica and original presentation by Diego Ongaro & John Ousterhout:

(http://www2.cs.uh.edu/~paris/6360/PowerPoint/Raft.ppt)

Raft

- Home https://raft.github.io/
  - http://thesecretlivesofdata.com/raft/
- Presentation
  - https://www.youtube.com/watch?v=LAqyTyNUYSY
  - https://www.youtube.com/watch?v=YbZ3zDzDnrw
    Watch it!
- Principles
  - Relies on RPC with **idempotent** operations
  - Maintains a replicated globally consistent command log
  - Clients must talk to the leader
- Non byzantine, based on dynamic leader elections (timer based)

Replicated State Machines



- **Replicated log** ⇒ replicated state machine
  - All servers execute same commands in same order
- Consensus module ensures proper log replication
- System makes progress as long as any majority of servers are up
- Failure model: fail-stop (not Byzantine), delayed/lost messages

Designing for understandability

- A main objective of RAFT
  - Whenever possible, select the alternative that is the easiest to understand
- Techniques that were used include
  - Dividing problems into smaller problems
  - Reducing the number of system states to consider
- Simple data model
  - **log of atomic immutable objects**
  - no znode structures like in Zookeeper

Raft Overview

1. Leader election
   - Select one of the servers to act as cluster leader
   - Detect crashes, choose new leader

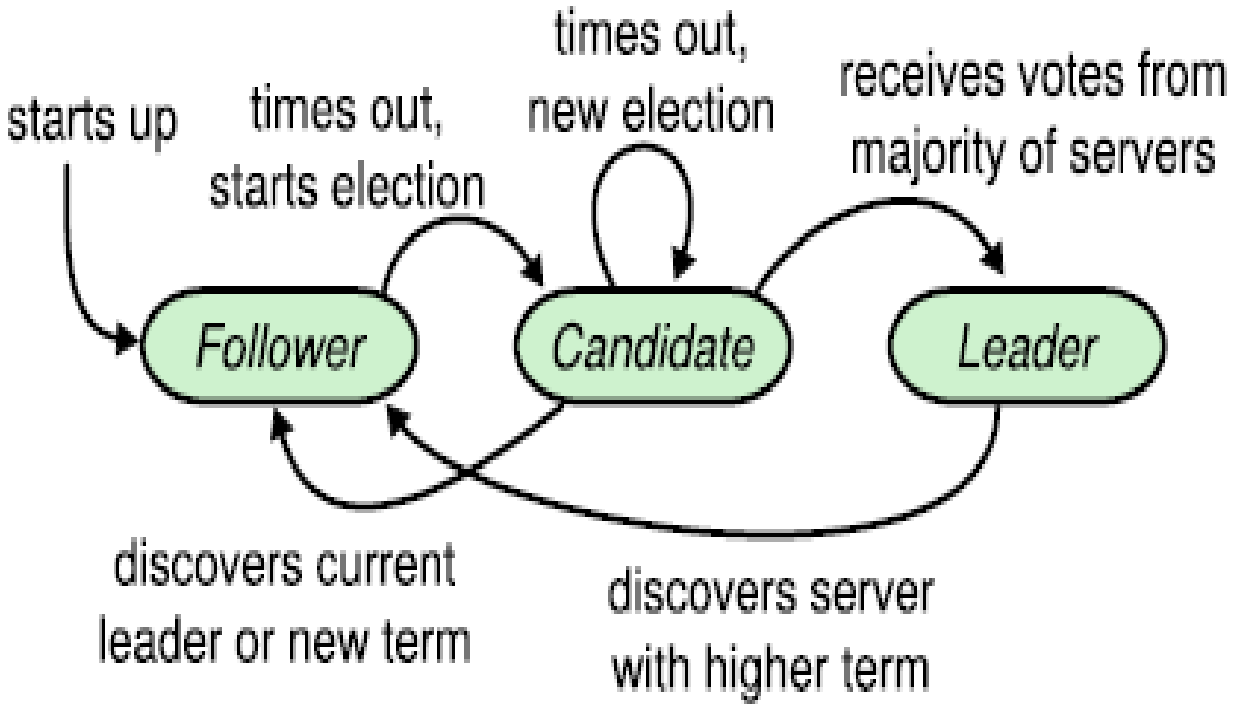
2. Log replication (normal operation)
   - Leader takes all commands from clients, appends them to its log
   - Leader replicates its log to other servers (overwriting inconsistencies)

3. Safety
   - Only a server with an up-to-date log can become leader

Raft basics: the servers

- A RAFT cluster consists of several servers
  - Typically, a small number (e.g., seven, five, or just three)
- Each server can be in one of three states
  - ***Leader***
  - ***Follower***
  - ***Candidate*** (to be the new leader)
- Followers are passive:
  - Simply reply to requests coming from their leader

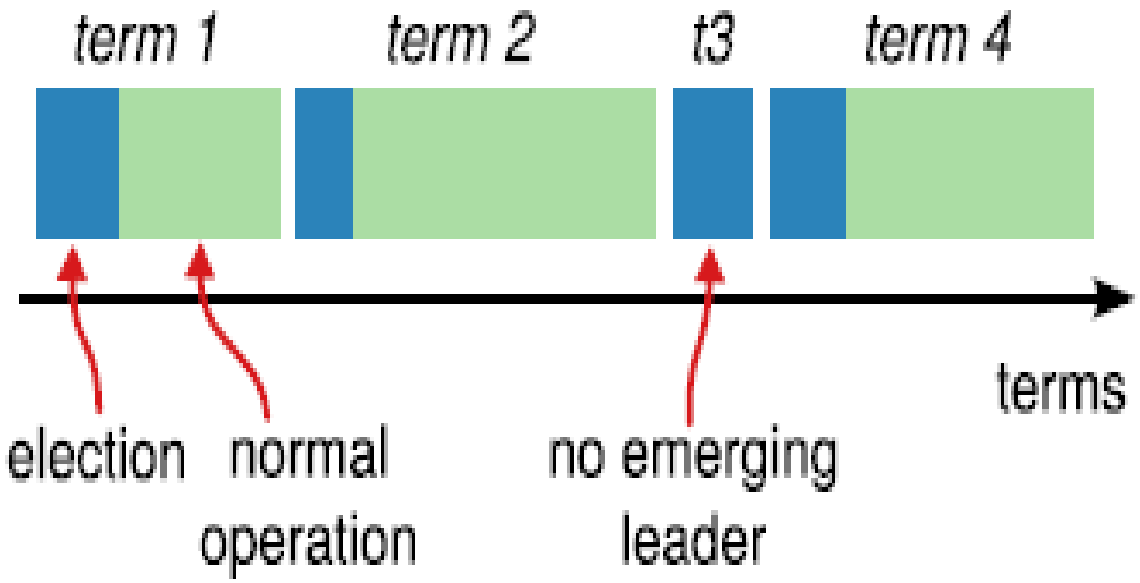
Server states



Raft basics: terms (I)

- Epochs of arbitrary length
  - Start with the election of a leader
  - End when
    - Leader becomes unavailable
    - No leader can be selected (split vote)
- Different servers may observe transitions between terms at different times or even miss them

Raft basics: terms (II)



Raft basics: RPC

- Servers communicate though idempotent RPCs

- **RequestVote**
  - Initiated by candidates during elections

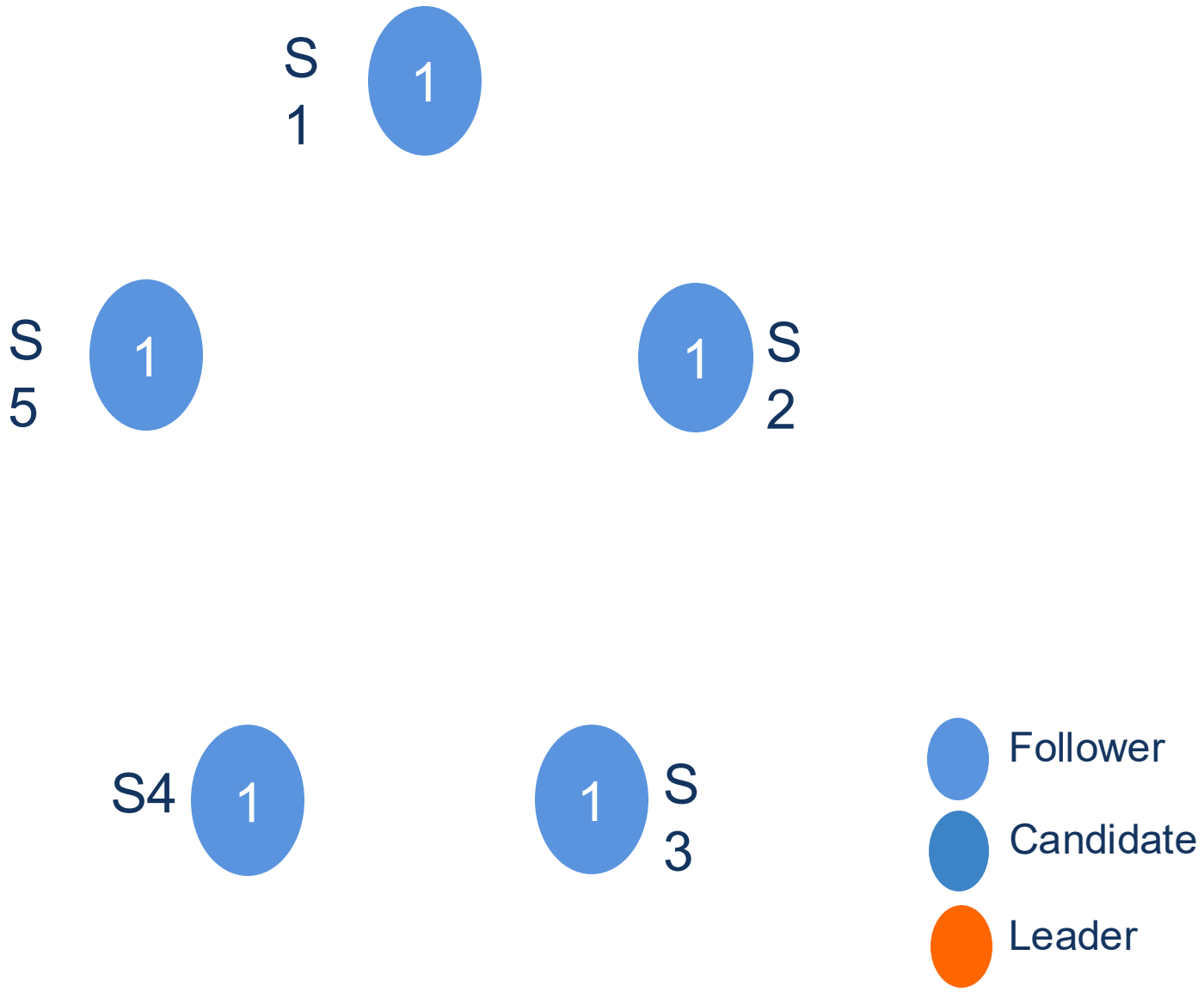
- **AppendEntry:** Initiated by leaders to
  - Replicate log entries
  - Provide a form of heartbeat
    - Empty AppendEntry( ) calls

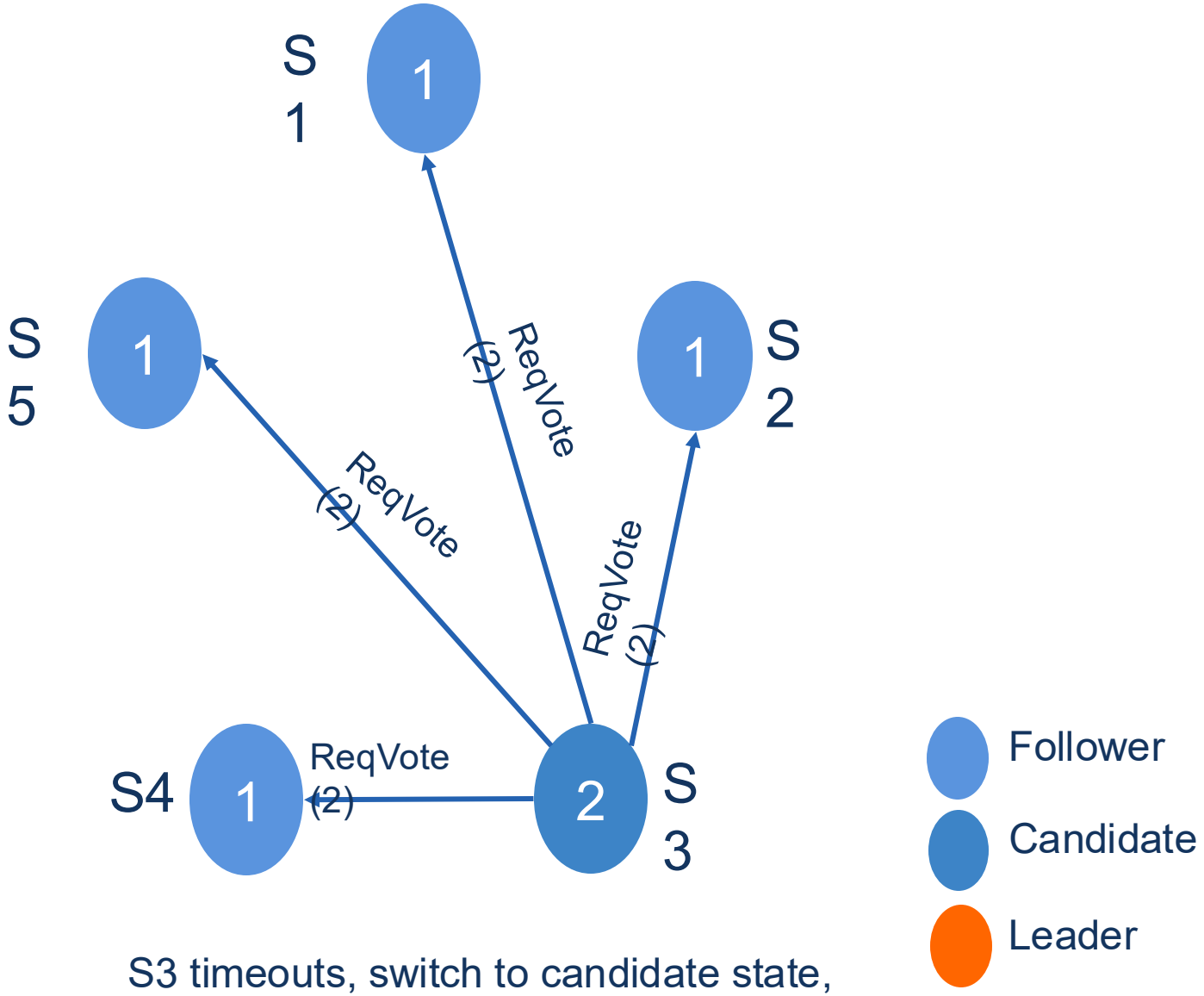
Leader elections

- Servers start being ***followers***

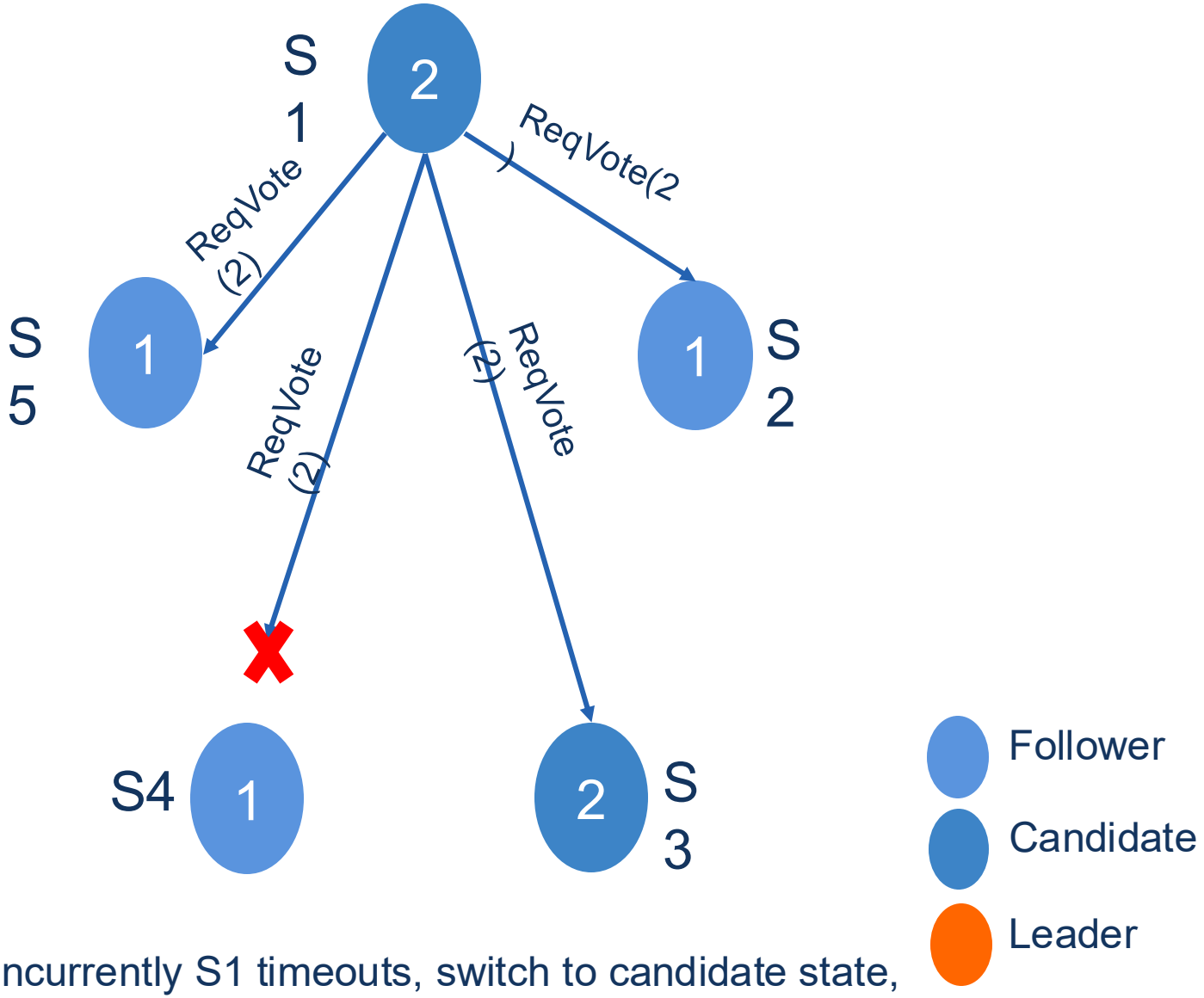
- Remain followers as long as they receive valid RPCs from a leader or candidate
  - If there is no new data, the leader still heart-beats

- When a follower receives no communication (data or heart-beats) over a period of time (the ***election timeout***), it starts an election to pick a ***new leader***

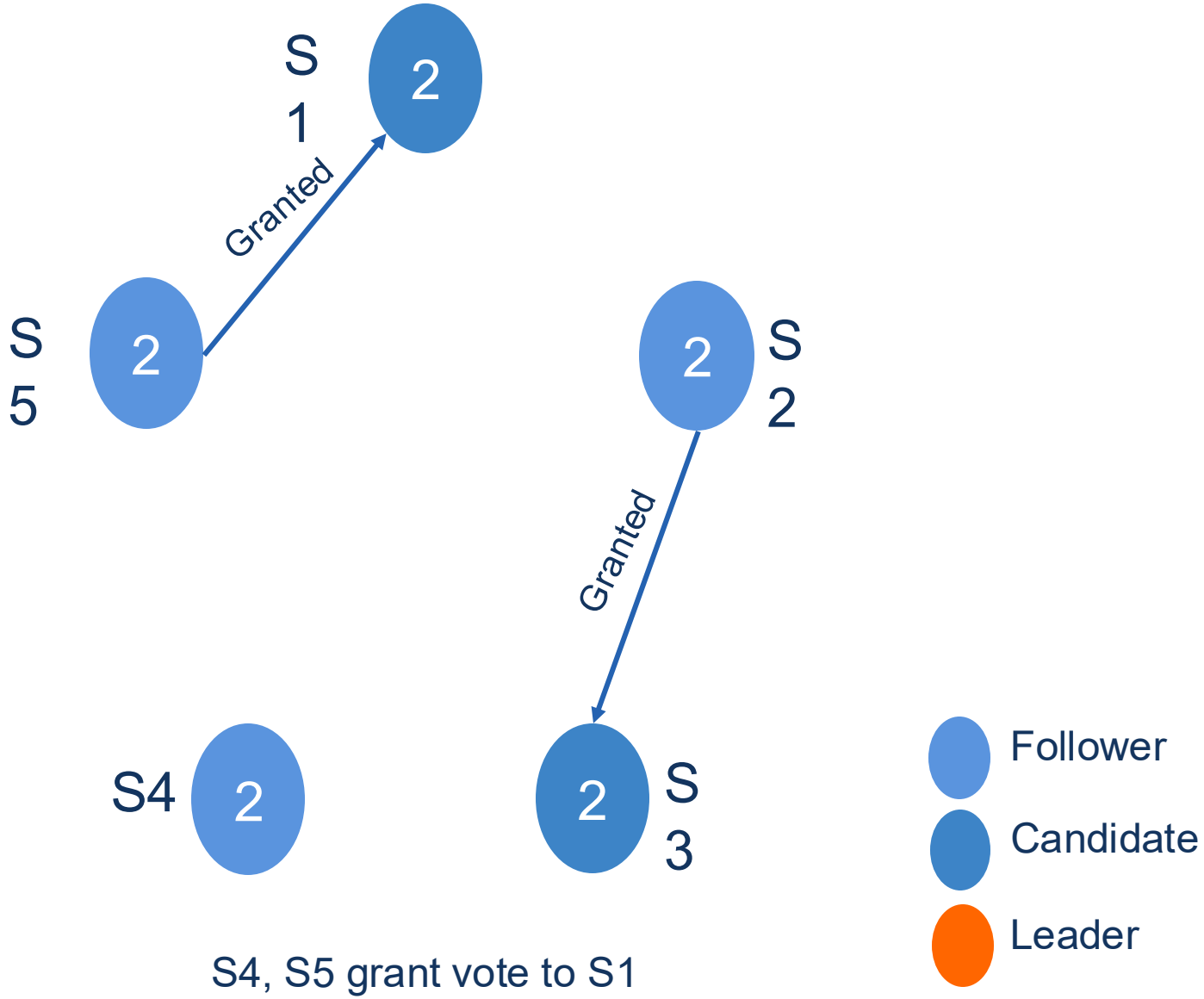

S
1
1
S
5
1
1
S
2
S4
1
1
S
3
Follower
Candidate
Leader



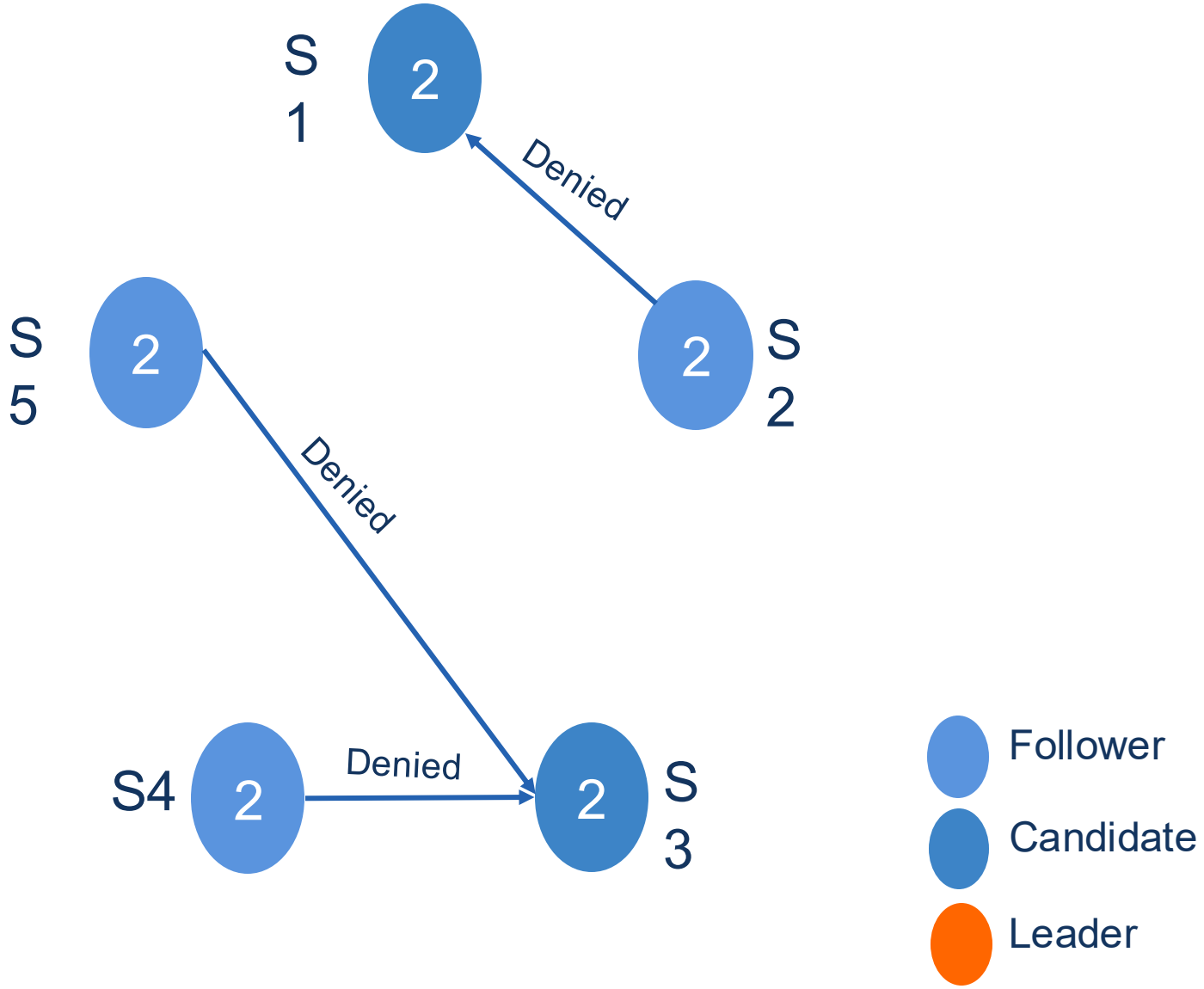
S3 timeouts, switch to candidate state,
increment term, vote itself as a leader and ask everyone else to confirm

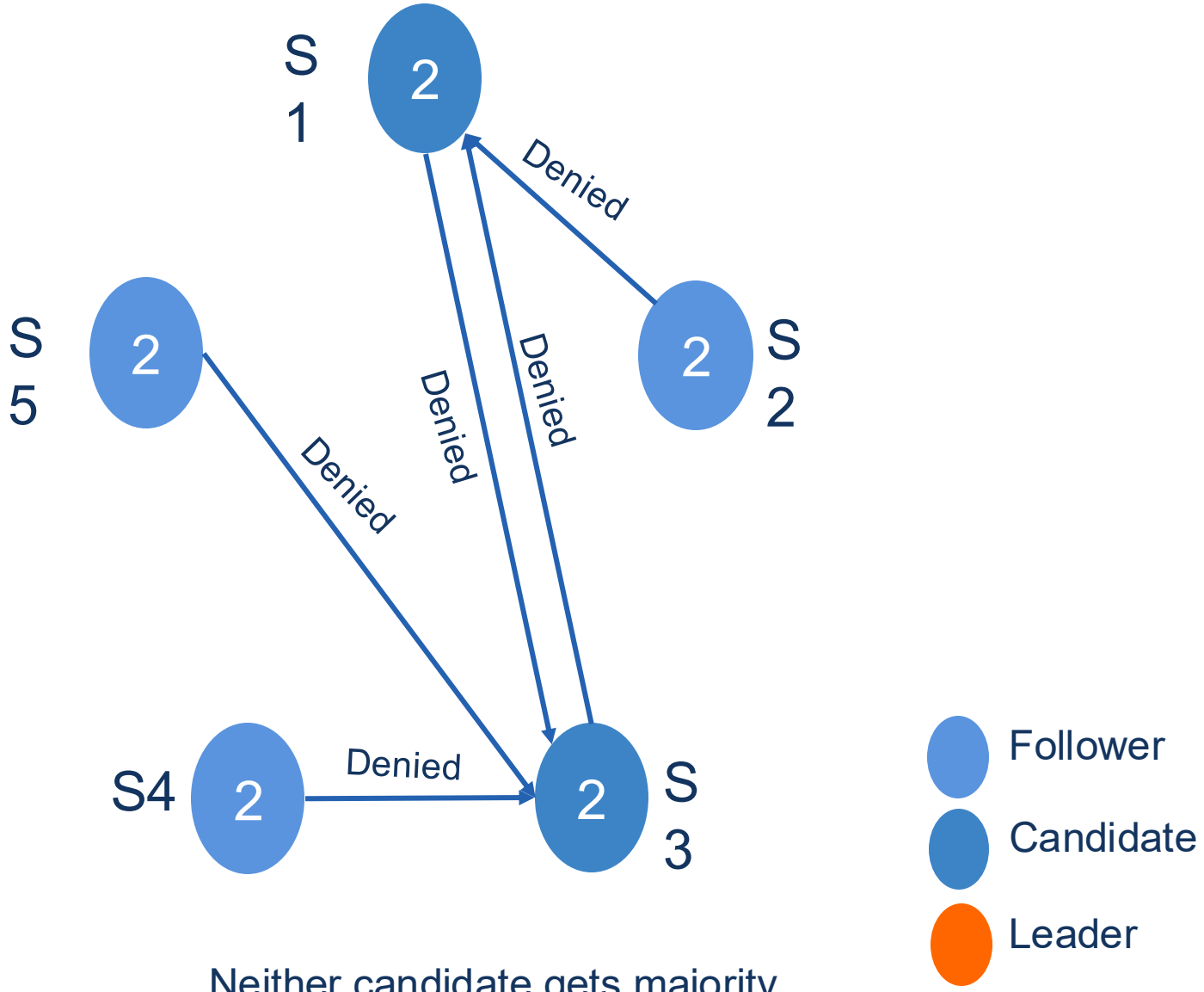


Concurrently S1 timeouts, switch to candidate state, increment term, vote itself as a leader and ask everyone else to confirm

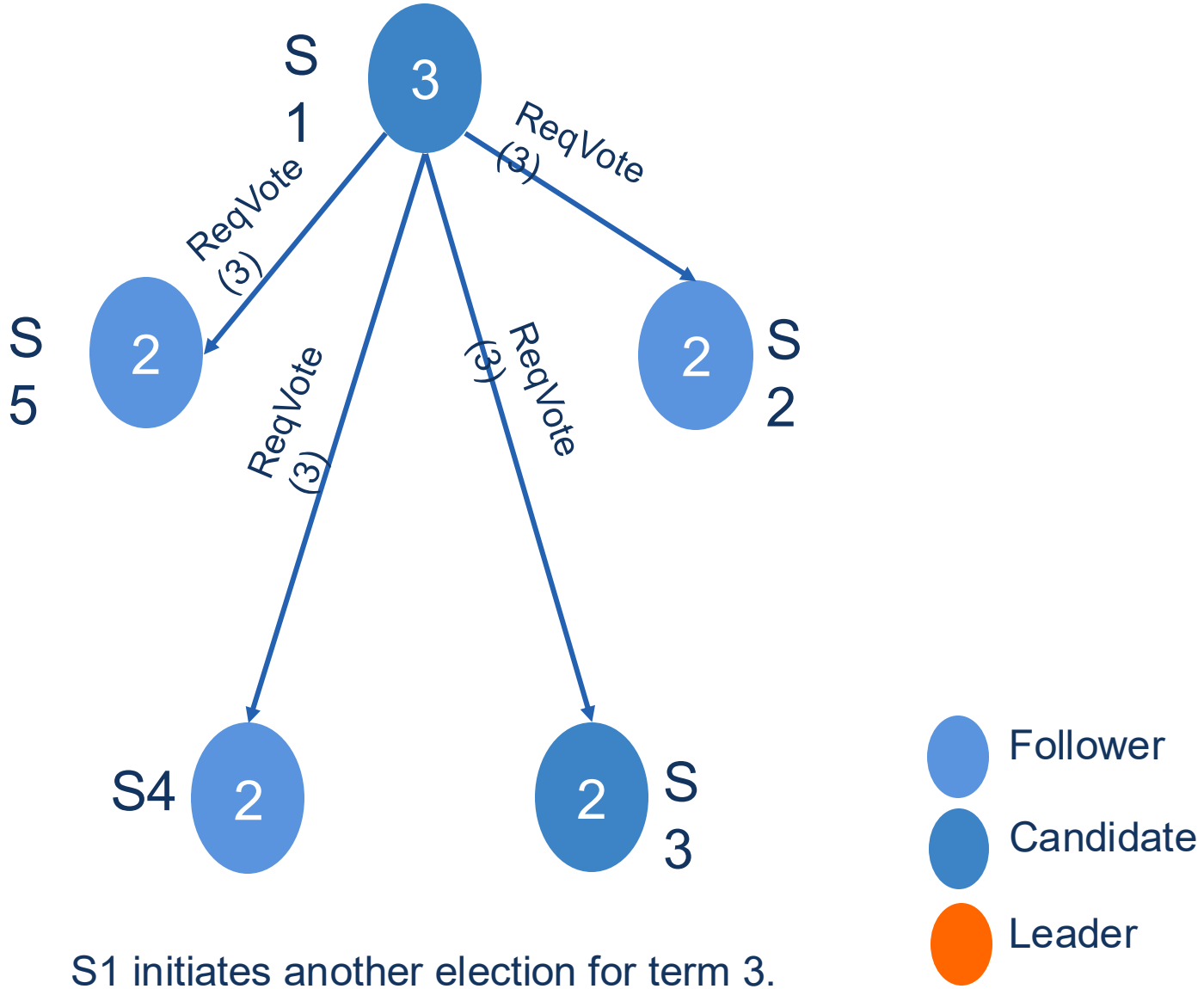


S4, S5 grant vote to S1
S2 grants vote to S3

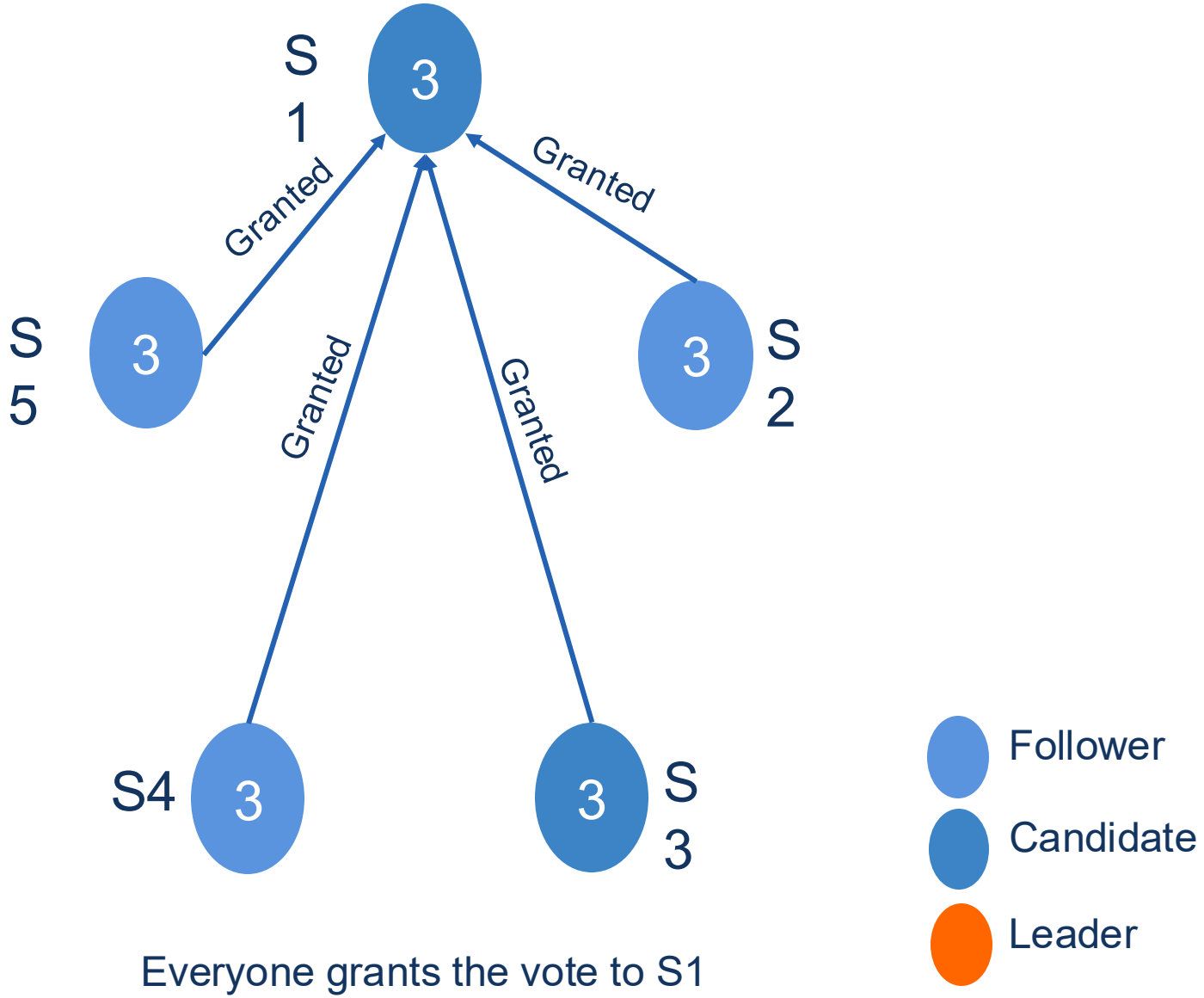

S
1
2
Denied
S
5
2
2
S
2
Denied
S4
2
Denied
2
S
3
Follower
Candidate
Leader



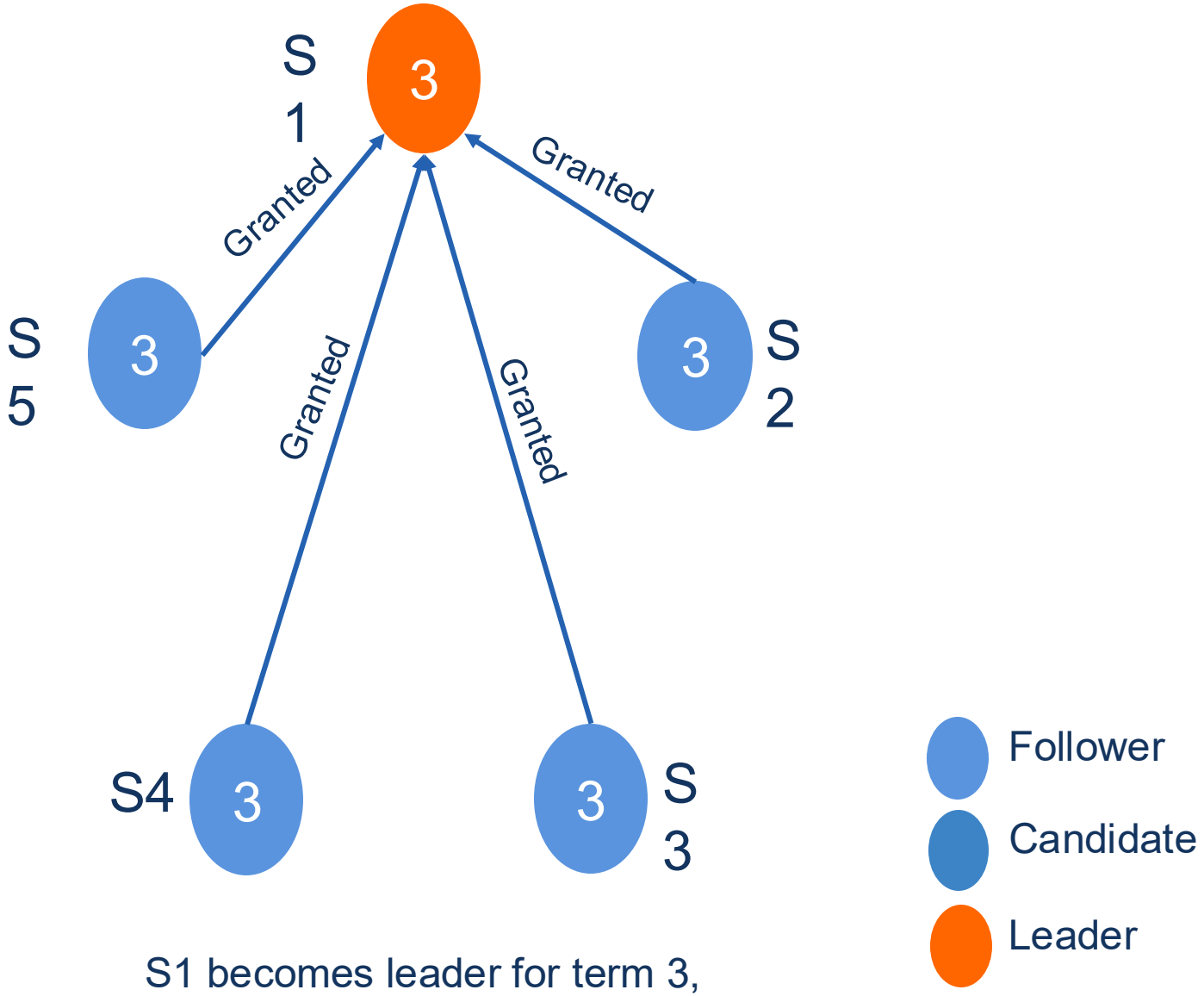
Neither candidate gets majority.
After a random delay between 150-300ms try again.



S1 initiates another election for term 3.



Everyone grants the vote to S1

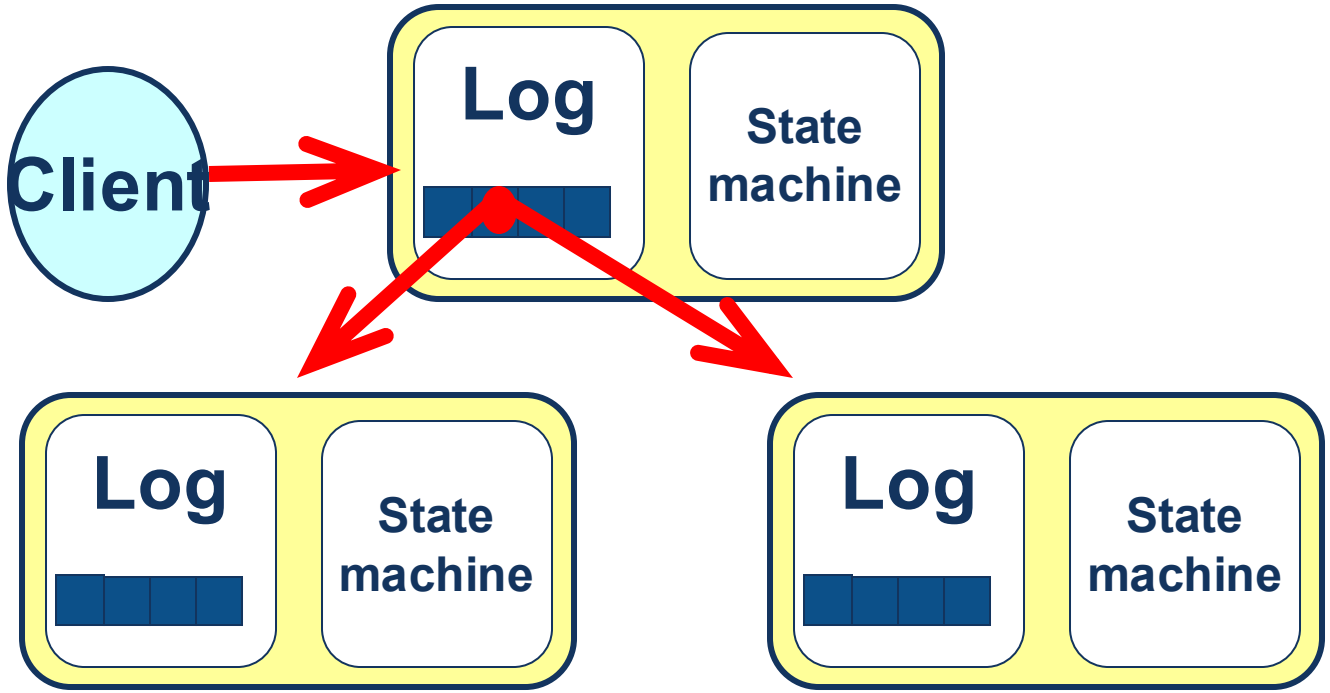


S1 becomes leader for term 3,
and the others become followers.

Log replication

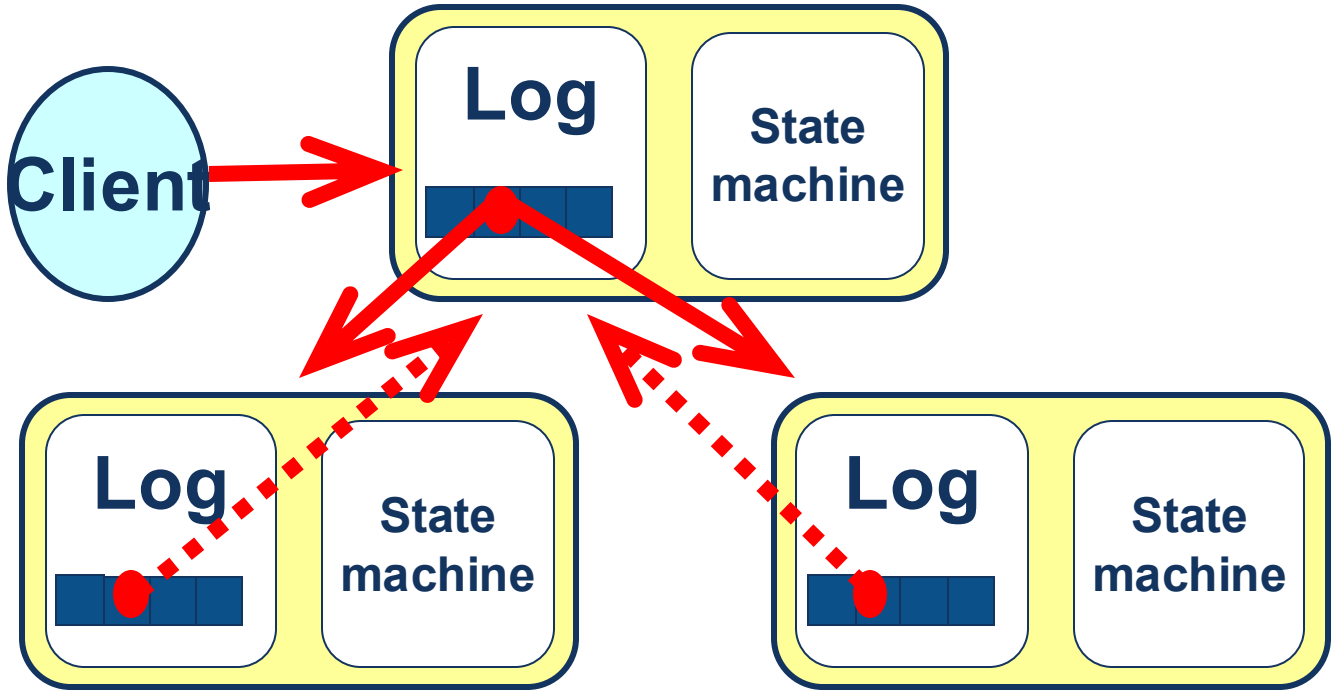
- Leaders
  - Accept client commands
  - Append them to their log (new entry)
  - Issue **AppendEntry** RPCs in parallel to all followers
  - Apply the entry to their state machine once it has been safely replicated
    - Entry is then ***committed***

A client sends a request



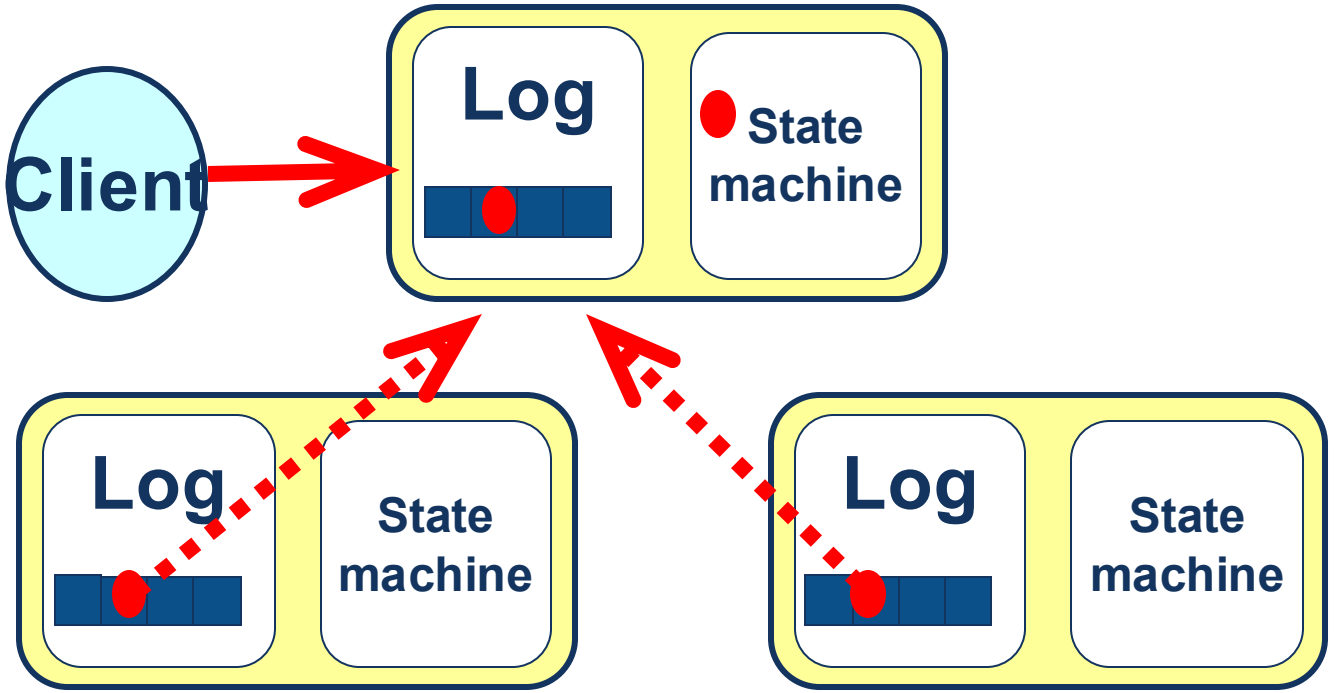
- Leader stores request on its log and forwards it to its followers

The followers receive the request



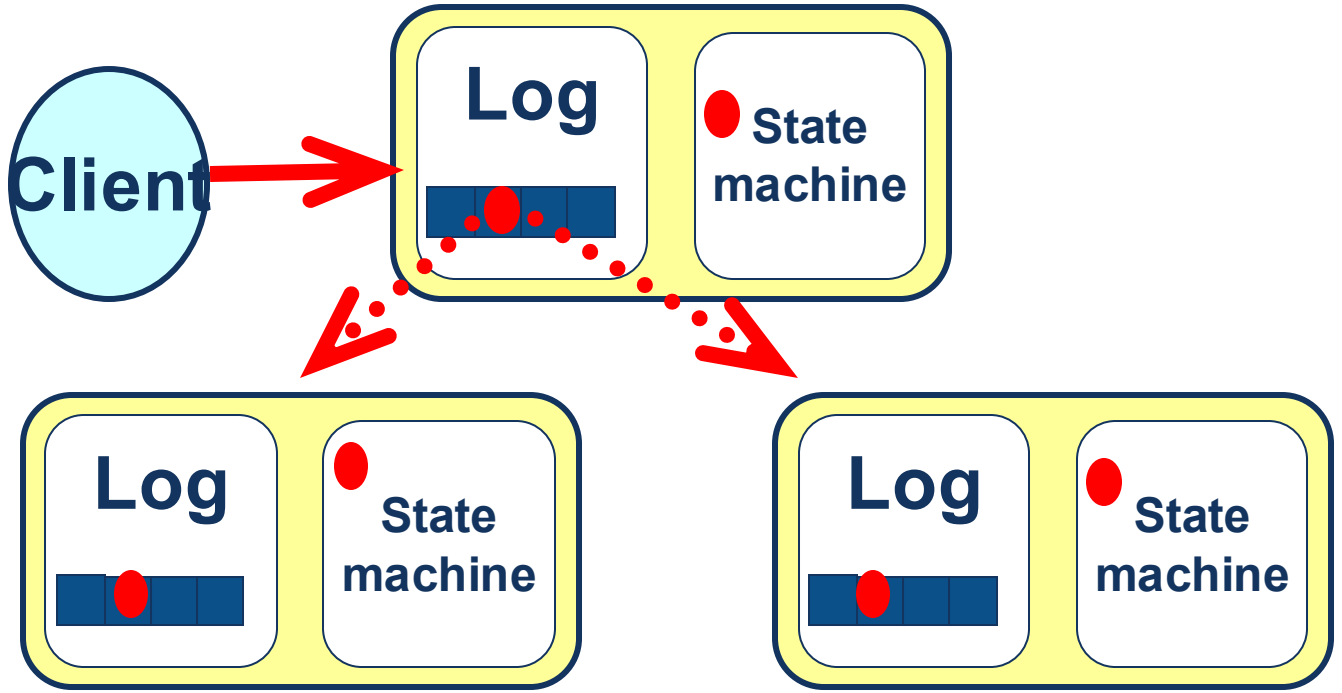
- Followers store the request on their logs and acknowledge its receipt

The leader tallies followers' ACKs



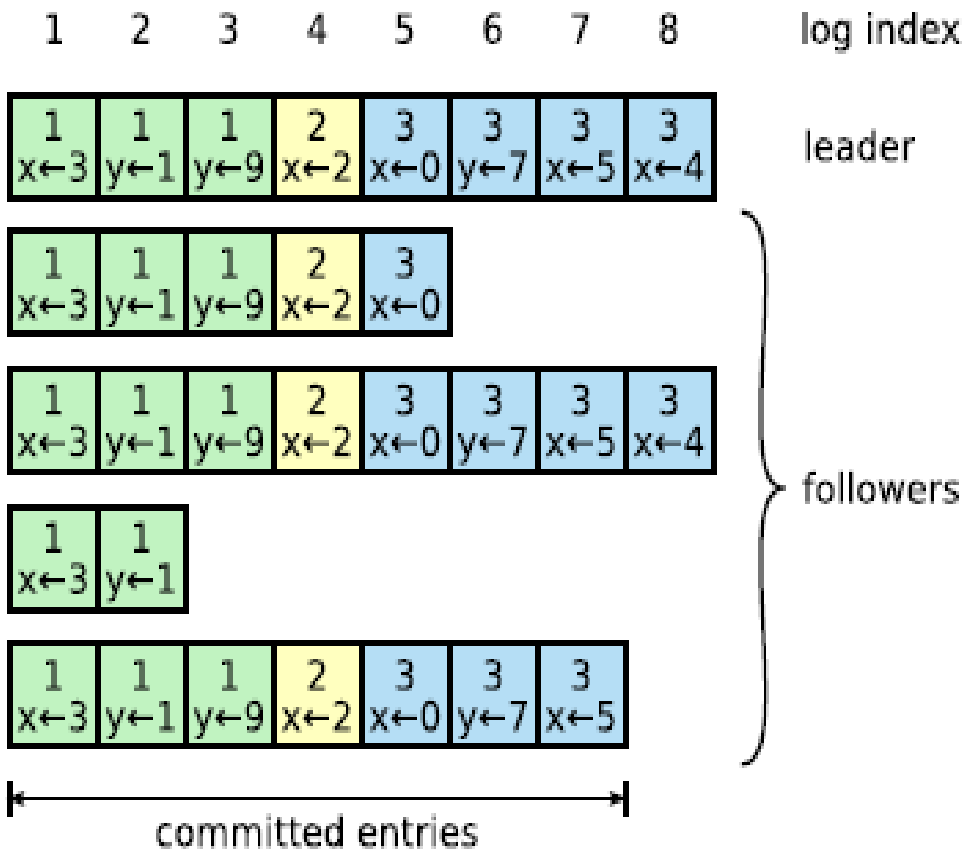
- Once it ascertains the request has been processed by a majority of the servers, it updates its state machine

The leader tallies followers' ACKs



- Leader's heartbeats convey the news to its followers: they update their state machines

Log organization



Colors identify terms

Handling slow followers ,…

- Leader reissues the AppendEntry RPC
  - They are idempotent

Committed entries

- Guaranteed to be both
  - Durable
  - Eventually executed by all the available state machine

- Committing an entry also commits all previous entries
  - All AppendEntry RPCs—including heartbeats—include the index of its most recently committed entry

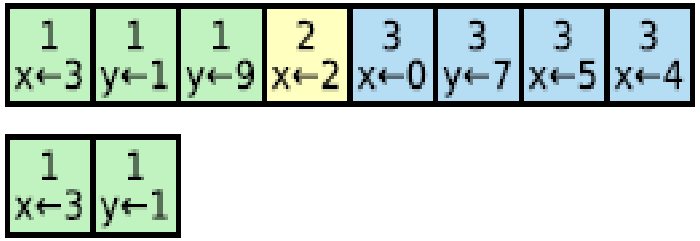
Why?

- Raft commits entries in ***strictly sequential order***
  - Requires followers to accept log entry appends in the same sequential order
    - ***Cannot "skip" entries***

Greatly simplifies the protocol

Raft log matching property

- If two entries in different logs have the same index and term
  - These entries store the same command
  - ***All previous entries*** in the two logs are ***identical***



Safety

- Two main questions

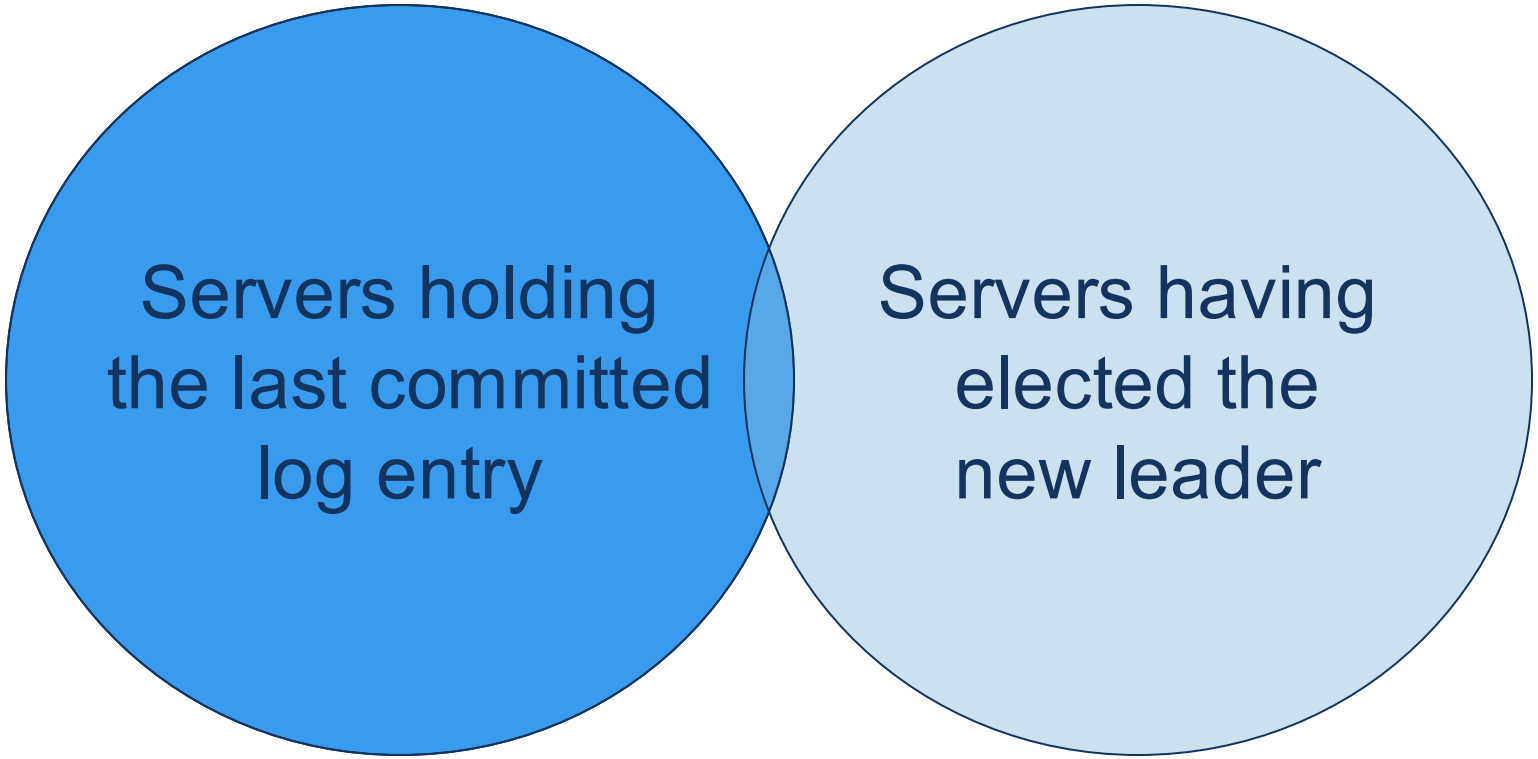
1. What if the log of a new leader did not contain all previously committed entries?
   - Must impose conditions on new leaders

2. How to commit entries from a previous term?
   - Must tune the commit mechanism

Election restriction (I)

- The log of any new leader ***must*** contain all previously committed entries
  - Candidates include in their ***RequestVote*** RPCs information about the state of their log
  - Before voting for a candidate, servers check that the log of the candidate is at least as up to date as their own log.
    - Majority rule does the rest

Election restriction (II)



Two majorities of the same cluster ***must*** intersect

Committing entries from previous term

- A leader cannot conclude that an entry from a previous term is committed even if stored on a majority of servers.
- A leader should never commit log entries from previous terms by counting replicas
- Should only do it for entries from the current term
- Once it has been able to do that for one entry, all prior entries are committed indirectly

Summary

- Consensus is a key building block in distributed systems
- Raft is similar to Paxos but
  - arguably easier to understand
  - elected leaders retain that role for epochs
  - **Idempotent updates**
  - separated stages reduces the algorithm state space
  - provides a detailed implementation
- Raft similar to Zookeeper but
  - **Replicated log** rather than replicated data container
  - Much simpler (less powerful) data model

Comparison with traditional databases

- Transaction Processing and Concurrency
  - Some databases provide ACID and strong data consistency, e.g., using master/leader
  - Some databases offer eventual consistency, great scalability, automatic replication and sharding
  - Typically, NO Consensus in either case

  - Like Kademlia, massively distributed data stores typically do not rely on global consensus
  - Others use consensus only on slowly changing system properties

Etcd

- Etcd is distributed key-value store that provides reliable storage across a cluster of nodes
- It uses raft consensus to manage an HA replicated log
- Should be used to store meta-data and configurations, e.g., to coordinate nodes
- Replicates all to all, no sharding or horizontal scale out
- More info here

  https://www.slideshare.net/slideshow/distributed-fun-with-etcd/126313792

Exercise (voluntary)

1. Explain with your own words and figures how Raft works, main principles
2. How is this different from Zookeeper?
3. What are your major takeaways on global consensus (in your own words)?

Discussion and Questions

